

Exercise Sheet 2

No Fear of Numbers: Introduction to Quantitative Data Analysis in R

Dr. Susanne de Vogel, Dr. Maryam Movahedifar, Data Science Center, University of Bremen

11 March 2025

Univariate Analysis

In the previous exercise, we got to know our dataset and some of the main variables. In this sheet, we focus on univariate analysis looking at **one variable at a time**. Univariate analysis is the starting point of any data analysis. It helps us **understand distributions**, typical values, and strange values and is the basis for all further steps: bivariate relationships and regression models.

In the context of our research story, we are interested in **overall life satisfaction** (blcd01) and **having children (yes/no)** (from blcd06). Before we relate these outcomes to other factors, we first want to understand

- How are these variables distributed in our sample?
- How are other key variables (e.g. gender, age, burden, work–life compatibility) distributed?

In this exercise, you will:

- transform categorical variables to factors
- describe nominal variables using absolute and relative frequencies, mode, and bar charts,
- describe ordinal variables using cumulative frequencies, median, percentiles,
- describe metric variables using mean, standard deviation, percentiles, boxplots, histograms, skewness and kurtosis, standard error, confidence intervals

0 Set-up: Load packages and dataset, load data frame

Install `tidyverse`, `haven`, `janitor`, `sjmisc`, `scales`, `skimr` and `moments` if not already installed and load these packages. Set the “Exercise” folder as your working directory and load dataset “02_qa_uni_data.sav” as a data frame called `mydata2`.

Code it (or faster: copy and paste the code) like this:

```

# Install tidyverse
if (!requireNamespace("tidyverse", quietly = TRUE))
  install.packages("tidyverse")

# Install haven
if (!requireNamespace("haven", quietly = TRUE))
  install.packages("haven")

# Install readxl
if (!requireNamespace("janitor", quietly = TRUE))
  install.packages("janitor")

# Install skimr
if (!requireNamespace("skimr", quietly = TRUE))
  install.packages("skimr")

# Install scales
if (!requireNamespace("scales", quietly = TRUE))
  install.packages("scales")

# Install moments
if (!requireNamespace("moments", quietly = TRUE))
  install.packages("moments")

# Install sjmisc
if (!requireNamespace("sjmisc", quietly = TRUE))
  install.packages("sjmisc")

## load packages
library(tidyverse)
library(haven)
library(sjmisc)
library(skimr)
library(janitor)
library(scales)
library(moments)

# set working directory
setwd(
  ↪ "C:/Users/Susanne/Nextcloud/share_DSC/003_Trainings/001_Workshops/2026/2026-03-11_SdV_MM_Quantitative_Data_Anal
  ↪ )

# Load the Nacaps data set and name this data frame "mydata2"
mydata2 <- read_sav("02_qa_uni_data.sav")

```

On the right side, in the Environment pane, the data frame `mydata2` should now appear under **Data**.

Exercise 1: Categorical variables: Children and gender

1.1. Calculate absolute and relative frequencies for `blcd06` children (1 = yes/ 2 = no).

Use the function `tabyl()` from the *janitor* package to create a frequency table for `blcd06`. Look at the counts and percentages in the output, what does it tell you?

1.2. Alternative: Calculate Absolute and relative frequencies for `adem01` gender *with labels*

`tabyl()` will show the **numeric codes**, which is not very intuitive. To make the table easier to read, we want to display the **variable and value labels** instead.

a. To do so, first convert `adem01` into a *factor* with labels.

Use `mutate()` and `adem01 = as_factor(adem01, levels="labels")` to transform `adem01` into a factor variable. Save the result again as `mydata2`. The code should look like this:

```
# convert the labelled variable into a factor whose levels are the value labels (e.g. "female",
  ↪ "male", ...)
mydata2 <- mydata2 %>%
  mutate(
    adem01 = as_factor(adem01, levels="labels")
  )
```

b. Create a frequency table for `adem01` using `tabyl()`.

Compare this to the table for `blcd06` with numeric codes. Decide which version is easier to read.

1.3. Identify the *mode* of `blcd06` and `adem01`

The mode is the category that appears most often. As there are variables with little categories, we can easily identify it by looking at the frequency tables we created with `tabyl()`.

1.4. Visualize the distribution of `blcd09` children (yes/no) in a bar chart.

Draw a bar chart with `blcd06` on the x-axis using `ggplot()` function. The height of the bars should show how many people are in each category (1/2). The code should look like this:

```
mydata2 %>% # take mydata2 as input
  ggplot(aes(x = blcd06)) + # put blcd06 on x-axis
  geom_bar() + # draw bars, counting rows per category
  labs( # add labels and title
    x = "Children (1 = yes, 2 = no)", # label for x-axis
    y = "Number of respondents", # label for y-axis
    title = "Distribution of having children (yes/no)" # main title of the plot
  )
```

What do you notice?

1.5. Create a similar bar chart for the distribution of gender `adem01`. What do you notice?

1.6. Modify the graph so 1.) missings do not appear and 2.) percentages are plotted.

To do so, you need to remove missing values out of `adem01`, count the cases in the category and use these information for plotting the bar chart. Modify the `ggplot()` function like this:

```
mydata2 %>%
  filter(!is.na(adem01)) %>% # remove missing values of adem01
  count(adem01) %>% # count how many cases in each category
  mutate(prop = n / sum(n)) %>% # compute proportion (relative frequency)
  ggplot(aes(x = adem01, y = prop)) + # map category to x-axis, proportion to y-axis
  geom_col() + # draw bars with given heights (prop)
  scale_y_continuous(labels = percent) + # scales package: show y-axis as percentages
  labs(
    x = "Gender", # x-axis label
    y = "Percent of respondents", # y-axis label
    title = "Share of genders in the sample"
  )
```

Exercise 2: Metric variables: Overall life-satisfaction

Our outcome variable `bpsy01` measures the **overall life satisfaction** rated on a scale from 0 to 10. Technically, it is an **ordinal variable**, but in research practices, scales like this are often **treated as metric**. And this is what we are going to do.

2.1. Look at the distribution of `bpsy01`. Calculate the *absolute, relative and cumulative frequencies*, read the *percentiles* and look at the *mean and standard deviation* using the `frq()` function.

2.2. Examine `bpsy01` using the `summary` and the `skim()` function from the *skimr* package. What can you see here? How do these functions differ?

2.3. To visualize the distribution of `bpsy01`, draw a boxplot with `ggplot`.

The code goes like this:

```
mydata2 %>%
  filter(!is.na(bpsy01)) %>%           # remove missings
  ggplot(aes(x = "", y = bpsy01)) +     # bpsy01 on y-axis
  geom_boxplot() +                     # draw boxplot
  labs(
    x = "",
    y = "bpsy01 (0-10 scale)",
    title = "Distribution of bpsy01 (boxplot)"
  )
```

What does the boxplot show you?

2.4. To see the overall shape of the distribution of `bpsy01`, create a *histogram* using `ggplot()`. Use `bpsy01` on the x-axis and choose a bin width of 1 to show the 0–10 scale clearly.

The code goes like this:

```
# histogram for bpsy01
mydata2 %>%
  filter(!is.na(bpsy01)) %>%           # remove missings
  ggplot(aes(x = bpsy01)) +             # bpsy01 on x-axis
  geom_histogram(binwidth = 1) +        # bins of width 1 (0,1,2,...,10)
  labs(
    x = "bpsy01 (0-10 scale)",
    y = "Number of respondents",
    title = "Distribution of bpsy01 (histogram)"
  )
```

What do you see regarding the skewness of `bpsy01`?

2.5. Let's look at inference statistics for `bpsy01`. Calculate *standard errors* and both *95 % confidence intervals* using the `summarize()` function.

Code it like this:

```
mydata2 %>%
  summarise(
    n = sum(!is.na(bpsy01)),           # number of valid cases
    mean_val = mean(bpsy01, na.rm = TRUE), # mean
    sd_val = sd(bpsy01, na.rm = TRUE),   # standard deviation
    se_val = sd_val / sqrt(n),           # standard error of the mean
    ci_lower = mean_val - 1.96 * se_val,  # 95% CI lower bound
    ci_upper = mean_val + 1.96 * se_val   # 95% CI upper bound
  )
```